You said:

exec(1).pyc
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 1m 43s

ChatGPT said:

Análise de segurança do arquivo `exec(1).pyc`

Veredito: MALWARE

Categoria principal: worm/botnet com força bruta de SSH e propagação automatizada entre servidores Linux.

Análise estática do bytecode Python, sem executar o arquivo.

O arquivo contém código projetado para descobrir servidores SSH acessíveis, testar senhas, registrar acessos bem-sucedidos, instalar serviços persistentes e tentar propagar uma carga para outras máquinas. Há também comunicação com um bot do Telegram e com servidores externos.

Ressalva importante: encontrei um defeito que provavelmente impede a implantação maliciosa quando este arquivo é executado diretamente na forma `.pyc` recebida. Isso não torna seu conteúdo benigno: significa que a amostra pode estar defeituosa ou incompleta.

1. Identificação do arquivo

A inspeção do cabeçalho e da estrutura interna revelou:

Propriedade	Resultado
Arquivo	`exec(1).pyc`
Formato	Bytecode Python compilado
Versão identificada	CPython 3.13
Tamanho	31.661 bytes

SHA-256	`a98cd601a21b8fb20bafcac729fa7d0ccab1c75bcfeb4bc8748447b78babf0f4`

O bytecode foi desserializado e inspecionado estaticamente, sem executar suas instruções. A estrutura contém várias rotinas Python e um script Bash armazenados como strings, além de funções de implantação e inicialização.

2. Análise do comportamento

Etapa 1 — Instalação com privilégios elevados

A função `main()` verifica se o programa está sendo executado como `root`.

O código contém comandos para instalar pacotes de sistema, incluindo `masscan`, `curl`, `screen` e componentes do Python. Em seguida, prepara arquivos em diretórios como `/root/.s/` e `/opt/monitor`.

Indicador suspeito: o programa exige privilégios administrativos para configurar uma infraestrutura de varredura e propagação.

Etapa 2 — Persistência por serviços do sistema

O programa prepara dois serviços do `systemd`:

- `scan-runner.service`: executa continuamente o processo de varredura.
- `file-monitor.service`: executa o componente responsável pelo monitoramento dos resultados e pela propagação.

As configurações usam `User=root`, habilitam a inicialização dos serviços e definem reinicialização automática em caso de falha.

Interpretação: esse mecanismo permite manter as tarefas maliciosas ativas após reinicializações e tentar recuperá-las quando os processos terminam inesperadamente.

Etapa 3 — Varredura automatizada de servidores

O script Bash embutido configura o `masscan` para procurar portas TCP **22 e 5522**, geralmente associadas ao acesso SSH ou a serviços SSH em portas alternativas.

Ele utiliza faixas de IP obtidas de um servidor externo e repete a varredura continuamente. A taxa configurada pode chegar a 20.000 pacotes por segundo, dependendo da quantidade de núcleos de CPU.

Os resultados são convertidos para registros de IP e porta e encaminhados para as próximas etapas.

Interpretação: é uma rotina de reconhecimento de alvos em escala, compatível com a infraestrutura de um botnet.

Etapa 4 — Tentativas de autenticação por força bruta

A função `worker()` utiliza a biblioteca `paramiko` para estabelecer conexões SSH. O programa testa senhas contra os alvos encontrados, usando `root` como usuário padrão e até 100 tarefas concorrentes por padrão.

Quando uma autenticação funciona, o endereço e a porta são gravados em arquivos de resultados.

Outro detalhe importante é que o código aceita automaticamente chaves de host SSH desconhecidas, sem verificar sua autenticidade.

Interpretação: a combinação de varredura em massa, tentativa de senhas e armazenamento de acessos válidos constitui evidência forte de atividade maliciosa.

Etapa 5 — Propagação para outras máquinas

O componente `monitor_script.py` monitora arquivos com os resultados das autenticações. Quando encontra um alvo, ele:

1. Consulta uma API externa para verificar se o servidor já foi processado.
2. Tenta acessar o servidor utilizando uma lista de senhas.
3. Envia um arquivo Python para o destino por SFTP.
4. Executa comandos remotamente para instalar Python, caso necessário, e executar o arquivo transferido.
5. Tenta instalar um agente de monitoramento remoto.
6. Registra os resultados e informa o sucesso ou a falha por meio do Telegram.

Essas operações estão presentes no código embutido, embora a execução efetiva dependa de condições ambientais e de autenticação.

Interpretação: a tentativa de replicar a carga em novos servidores caracteriza um comportamento de **worm**, ou malware com propagação automatizada.

Etapa 6 — Comunicação externa e exposição de credenciais

O programa contém referências a:

- Uma API central no domínio `sou.pp.ua`.
- URLs de distribuição de arquivos e listas de IPs.
- A API de bots do Telegram.
- Um token e um identificador de chat incorporados ao bytecode.

O relatório enviado ao Telegram inclui o endereço do servidor-alvo, o usuário e a senha utilizados, além dos resultados das operações.

Isso significa que credenciais de servidores comprometidos podem ser transmitidas para um destino externo.

Interpretação: há evidências de comunicação para coordenação da operação e de divulgação de credenciais. O código também tenta apagar arquivos intermediários e excluir o próprio instalador após a implantação.

3. Limitação importante encontrada

Existe um defeito que afeta a execução desta amostra específica.

A função `main()` tenta abrir o próprio arquivo, por meio de `__file__`, como texto UTF-8. Entretanto, o arquivo recebido é um `.pyc` binário, cujo cabeçalho começa com bytes que não formam texto UTF-8 válido.

Essa leitura deve gerar uma exceção, que o próprio programa captura antes de retornar da função. Portanto, **se este arquivo for executado diretamente como o `.pyc` recebido, a implantação provavelmente será interrompida antes de instalar os serviços ou iniciar a varredura**.

Isso não elimina os indicadores maliciosos presentes no bytecode. Apenas significa que a amostra pode estar defeituosa ou que o código foi originalmente concebido para ser executado em sua forma-fonte `.py`.

4. Conclusão final

Classificação: MALWARE

Categoria principal: worm/botnet de força bruta SSH.

Comportamentos secundários identificados: varredura de rede, coleta e transmissão de credenciais, execução remota de comandos, propagação automatizada, persistência via `systemd` e comunicação com infraestrutura externa.

Confiança na classificação do conteúdo: alta.

Ressalva: a capacidade de executar essas ações a partir desta cópia `.pyc` está comprometida pelo defeito na leitura do próprio arquivo.

Não há necessidade de executar o arquivo para chegar a essa classificação: o conjunto de comportamentos está explícito nas rotinas embutidas.

Recomendação: não execute o arquivo em um sistema real. Se ele já foi executado a partir de uma versão-fonte correspondente, investigue os serviços e diretórios identificados, isole o sistema afetado e revogue o token do Telegram e quaisquer senhas reais presentes na amostra.

Também posso criar ↳ um diagrama do fluxo do malware para facilitar a visualização da cadeia de infecção.

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal deste malware é **Worm (verme de computador) com características de botnet**, especializado na **propagação por força bruta de SSH**.

Ele varre servidores em busca de portas SSH abertas, tenta autenticar-se com senhas, coleta credenciais válidas e tenta se propagar para outras máquinas, mantendo persistência por meio de serviços `systemd`.